Administração de Bases de Dados

Engenharia Informática – Universidade do Minho

Trabalho Prático – 2025/2026

Os grupos de trabalho devem ser constituídos por 4 elementos, todos inscritos na Unidade Curricular. O resultado do trabalho é um relatório escrito e código/scripts/resultados criados por cada grupo. O relatório deve omitir considerações genéricas sobre as ferramentas utilizadas, focando a apresentação e justificação dos objetivos atingidos. O desenvolvimento e entrega do trabalho é realizado em repositórios *git* disponibilizados pela equipa docente. O relatório deve identificar também o nome e número de todos os elementos do grupo. A data limite é 2026/01/04.

1 Contexto

O trabalho prático consiste na configuração, otimização, e avaliação de um *benchmark* disponível em cada repositório, baseado num pequeno excerto do dataset de metadados da plataforma *GitHub* e contendo operações transacionais e analíticas.

A componente transacional simula operações de *backend* do *GitHub*, considerando os seguintes procedimentos:

- **CreateRepository** – criar um novo repositório.
- **StarRepository** – adicionar uma estrela a um repositório.
- **CreateCommit** – criar um novo commit.
- **CreateTicket** – criar um novo *ticket* (*issue* ou *pull request*).
- **CloseTicket** – fechar um *ticket*.
- **GetRepositoryInfo** – obter a informação de um repositório.
- **GetRepositoriesByLanguage** – obter os repositórios mais recentes com uma dada linguagem de programação.
- **GetUserInfo** – obter o perfil de um utilizador.
- **GetTicketInfo** – obter a informação de um *ticket*.
- **SearchRepositories** – pesquisar repositórios através da descrição.

As quatro interrogações analíticas simulam operações estatísticas e de *data warehousing* sobre o dataset, definindo os seguintes argumentos:

- **Q1** – *n/a*.
- **Q2** – *mês* (valor por defeito = '2024-12-01').
- **Q3** – *intervalo de datas* (valor por defeito = '2022-01-01' a '2025-01-01') e *resolução de tempo* (valor por defeito = '15 minutes').

- **Q4** – *nome de utilizador* (valor por defeito = 'christopherharmon25').

Apesar dos argumentos estarem definidos de forma estática, deve-se considerar que podem ser modificados para **valores arbitrários**. Para além disso, é pretendido que as interrogações obtenham resultados sobre os **dados mais recentes**, sendo que não é viável a sua completa materialização.

2 Objetivos

A configuração de referência é uma máquina virtual na *Google Cloud* do tipo N2, 8 vCPUs, 16 GB RAM, disco Permanente SSD 500 GB (SSD Persistent Disk)[1], e sistema operativo Linux Ubuntu 25.04 x86/64. A carga transacional deve considerar 16 clientes, enquanto que a carga analítica deve considerar apenas 1. Usando esta configuração, devem:

1. Otimizar o desempenho da carga transacional (PostgreSQL).
2. Otimizar o desempenho das interrogações analíticas (PostgreSQL). Comparar e discutir o plano da versão otimizada em PostgreSQL com o plano sem otimizações em DuckDB, bem como os tempos de execução.

A realização destes objetivos deve considerar **redundância** (e.g., índices, materialização manual ou através de vistas materializadas), **paralelismo**, **parâmetros de configuração**, e até **código SQL/Java**.

Importante: Para a componente transacional, devem primeiro otimizar através da redundância e/ou código e só depois considerarem a otimização através dos parâmetros de configuração, de modo a obterem diferenças mais significativas neste último ponto.

3 Código e Dados

O código que implementa as operações descritas, bem como scripts de auxílio para carregar os dados, estão disponíveis em cada repositório. O dataset com os dados, em formato CSV, é disponibilizado em `https://storage.googleapis.com/abd_data/gitbench_data.zip`. Não coloquem o dataset no repositório!

4 Notas

- Como medidas de desempenho da carga transacional deve considerar-se principalmente o **débito máximo** atingível. Nas operações analíticas deve considerar-se o **tempo de resposta** médio de várias execuções.
- Poderão modificar o código SQL/Java e/ou o esquema da base de dados. Devem explicar no relatório em que medida essas alterações preservam o funcionamento da aplicação original.
- Devem considerar que cada *workload* corre separadamente, não sendo necessário executar a carga transacional e analítica de forma concorrente. Contudo, devem verificar que impacto a criação de redundância / alteração do esquema num *workload* tem no outro.
- Em cada um dos objetivos, poderão não ter tempo para considerar todas as otimizações possíveis nas suas várias combinações. Devem focar-se nas que consideram mais prometedoras e que mais vos interessam, justificando no relatório essas opções.
- No caso de não obterem melhorias de desempenho, devem explicar porque é que a configuração de referência já era ótima. Por outro lado, a simples apresentação de uma melhoria de desempenho, não justificada, não é muito interessante.

[1] Ao escolher o disco poderá ser necessário selecionar "Request quota adjustment" na altura da criação da máquina.

- A utilização de ferramentas de monitorização e diagnóstico do PostgreSQL e do sistema operativo (e.g., `pgbadger`, `psutil`, `iostat`) valoriza o trabalho.
- Devem considerar uma instalação nativa para o PostgreSQL (i.e., sem Docker).
- A utilização de ferramentas de IA é permitida quando: 1) usada como referência genérica para esclarecer conceitos (i.e., nada do enunciado é usado na prompt), caso em que o resultado também não deve aparecer no relatório; 2) usada para obter respostas sobre o enunciado (incluindo o código Java e SQL), caso em que a prompt deve constar integralmente no relatório, bem como a interpretação e validação dos resultados obtidos.
- A **automatização** da instalação, execução, e obtenção de dados do *benchmark* permitirá obter resultados em maior quantidade e uma análise mais profunda, valorizando o trabalho.
- Devem também procurar estratégias para poupar recursos na *Google Cloud*, por exemplo, armazenando os dados em *Cloud Storage* e reutilizando-os[2], em vez de re-executar o `load.py`, e/ou explorando o uso de *Machine Images*. Considerem as opções mais baratas na *Google Cloud* (i.e., regiões nos EUA, instâncias *spot*, ...) e de **não deixar máquinas virtuais ativas a consumir recursos desnecessariamente!**

[2] Ver `pg_dump` (https://www.postgresql.org/docs/18/app-pgdump.html) e `pg_restore` (https://www.postgresql.org/docs/18/app-pgrestore.html).